

Student

In SQL 1.1 I learned to filter and sort rows. But what if I want a single number — like the total revenue across all orders, or the number of customers per city?

ChatGPT • AI Assistant

CONCEPT CARD

That is exactly what **aggregate functions** are for. An aggregate function collapses multiple rows into a single computed value.

The five core aggregate functions:

- **COUNT** — how many rows?
- **SUM** — what is the total?
- **AVG** — what is the average?
- **MIN** — what is the smallest value?
- **MAX** — what is the largest value?

🔗 ANALOGY

Think of a spreadsheet column of numbers. An aggregate function is like pressing the SUM button at the bottom — it reads all the rows and produces one answer. GROUP BY is like grouping rows by category first and pressing SUM for each group separately.

🌐 REAL WORLD APPLICATION

Aggregations are the engine behind every dashboard, report, and analytics query in production. 'Total revenue this month', 'Average order value by city', 'Number of active users per day' — all of these are aggregate queries.

Student

And we are using the same e-commerce schema from SQL 1.1?

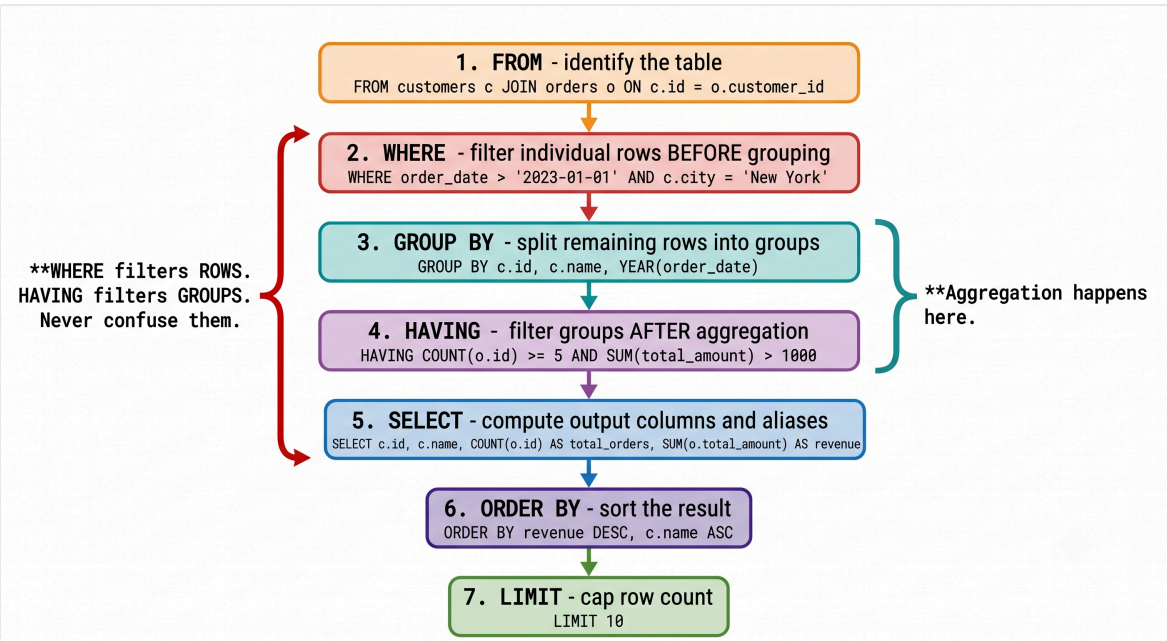
ChatGPT • AI Assistant

CONCEPT CARD

Exactly the same schema:

- `customers(customer_id, name, city, signup_date)`
- `orders(order_id, customer_id, order_date, total_amount, status)`
- `products(product_id, name, category, price, stock_qty)`

Every example in this section builds on what you already know from SQL 1.1.



SQL Full Logical Execution Order with GROUP BY and HAVING

Aggregate Functions Quick-Reference Cheat Sheet

Function	What It Does	NULL Behaviour	Example	Result
COUNT(*)	Counts all rows including NULLs	Includes NULL rows	COUNT(*)	Total row count
COUNT(col)	Counts non-NULL values in column	Skips NULL values	COUNT(status)	Non-NULL status count
COUNT(DISTINCT col)	Counts unique non-NULL values	Skips NULLs, deduplicates	COUNT(DISTINCT city)	Unique city count

Function	What It Does	NULL Behaviour	Example	Result
SUM(col)	Adds all non-NULL values	Ignores NULLs	SUM(total_amount)	Total revenue
AVG(col)	Average of non-NULL values	Ignores NULLs — denominator excludes NULLs	AVG(total_amount)	Mean order value
MIN(col)	Smallest non-NULL value	Ignores NULLs	MIN(price)	Cheapest product
MAX(col)	Largest non-NULL value	Ignores NULLs	MAX(total_amount)	Largest single order

sql – Full Query Template – All Clauses Together

```
-- Full execution order template
SELECT    city,
          COUNT(*)           AS total_orders,
          SUM(total_amount)  AS revenue,
          AVG(total_amount)  AS avg_order_value
FROM      orders o
JOIN      customers c ON o.customer_id = c.customer_id
WHERE     o.status ≠ 'cancelled'           -- filters rows BEFORE
grouping
GROUP BY  city                  -- splits into one group per
city
HAVING    SUM(total_amount) > 5000        -- filters groups AFTER
aggregation
ORDER BY  revenue DESC
LIMIT     5;
```

THE FIVE CORE AGGREGATE FUNCTIONS — COUNT, SUM, AVG, MIN, MAX

ChatGPT • AI Assistant

A side-by-side comparison diagram showing a sample orders table with 6 rows. The status column has values: 'shipped', 'delivered', NULL, 'pending', NULL, 'shipped'. Left panel (blue background, label 'COUNT(*) = 6'): all 6 rows are highlighted in green — every row is counted including the two NULL rows. Right panel (orange background, label 'COUNT(status) = 4'): only the 4 non-NULL rows are highlighted in green; the 2 NULL rows are grayed out with a strikethrough and label 'skipped — NULL'. A formula box at the bottom reads: 'COUNT(*) = all rows. COUNT(col) = non-NULL rows only. COUNT(DISTINCT col) = unique non-NULL values.' Warning icon next to the NULL rows with text: 'NULLs are silently excluded from COUNT(col) — this is a frequent source of bugs.'

COUNT(*) vs COUNT(column) — NULL Handling Difference

sql – All Five Aggregate Functions – Working Examples

```
-- COUNT variants
SELECT COUNT(*)           AS all_orders,
       COUNT(status)      AS orders_with_status,
       COUNT(DISTINCT status) AS unique_statuses
FROM orders;
-- Output: 4821 | 4803 | 4

-- SUM: total revenue
SELECT SUM(total_amount) AS total_revenue
FROM orders;
-- Output: 7284350.00

-- AVG: average order value (ignores NULLs in numerator AND denominator)
SELECT AVG(total_amount) AS avg_order_value
FROM orders;
-- Output: 1510.25

-- MIN and MAX together
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products;
-- Output: 99.00 | 4999.00

-- All five in one query with a WHERE filter
SELECT COUNT(*)           AS total_orders,
       SUM(total_amount)  AS total_revenue,
       AVG(total_amount)  AS avg_value,
       MIN(total_amount)  AS smallest_order,
       MAX(total_amount)  AS largest_order
```

```
FROM orders
WHERE status = 'delivered';
```

Student

Let's start simple. How do I count how many rows are in the orders table?

ChatGPT • AI Assistant

CONCEPT CARD

Use **COUNT(*)** — the asterisk means count every row regardless of NULL values.

sql — Code Example

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Output:

code — Code Example

```
total_orders
-----
4821
```

Student

What is the difference between COUNT(*) and COUNT(column)?

ChatGPT • AI Assistant

This is a classic interview question. The difference is in how NULLs are handled:

- **COUNT(*)** counts every row — including rows where some columns are NULL.
- **COUNT(column)** counts only rows where that specific column is NOT NULL.

EXAMPLE CARD

Example 1 — COUNT() vs COUNT(column)*

sql – Code Example

```
-- Total rows in orders table (includes rows where status is NULL)
SELECT COUNT(*) AS all_orders
FROM orders;
-- Result: 4821

-- Only rows where status is NOT NULL
SELECT COUNT(status) AS orders_with_status
FROM orders;
-- Result: 4803 (18 rows had NULL status – excluded)

-- Unique non-NULL status values
SELECT COUNT(DISTINCT status) AS unique_statuses
FROM orders;
-- Result: 4 (pending, shipped, delivered, cancelled)
```

 WARNING

If you want to count rows and expect a certain number but get a different count — check whether you used `COUNT(col)` on a column that has NULLs. Swap to `COUNT(*)` if you want all rows.

Example 2 — SUM: Total revenue from all orders

sql – Code Example

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
-- Result: 7284350.00
```

SUM ignores NULLs — rows with NULL total_amount are not added.

Example 3 — AVG: Average order value

sql – Code Example

```
SELECT AVG(total_amount) AS avg_order_value
FROM orders;
-- Result: 1510.25
```

 WARNING

AVG ignores NULLs in BOTH the numerator AND denominator:



$$\text{AVG} = \frac{\sum(\text{non}\backslash\text{text}\{-\}\text{NULL}\backslash;\text{values})}{\text{count}(\text{non}\backslash\text{text}\{-\}\text{NULL}\backslash;\text{rows})}$$

This means AVG(col) is NOT the same as SUM(col) / COUNT(*) when NULLs exist.

Example 4 — MIN and MAX: Price range of products

sql – Code Example

```
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive,
       MAX(price) - MIN(price) AS price_range
FROM products;
-- Output:
-- cheapest | most_expensive | price_range
-- 99.00    | 4999.00        | 4900.00
```

Example 5 — Multiple aggregates in one query

sql – Code Example

```
SELECT COUNT(*)           AS total_orders,
       SUM(total_amount)   AS total_revenue,
       AVG(total_amount)   AS avg_value,
       MIN(total_amount)   AS smallest_order,
       MAX(total_amount)   AS largest_order
FROM orders
WHERE status = 'delivered';
```

Output:

code – Code Example

total_orders	total_revenue	avg_value	smallest_order	largest_order
1823	2941050.00	1613.30	150.00	9800.00

Student



Why does AVG ignore NULLs in the denominator? Shouldn't it divide by the total

row count?

ChatGPT • AI Assistant

SQL's design choice: AVG is meant to represent the average of known values. If a row has NULL total_amount, we don't know the order value — so it should not drag the average down. Consider: 5 orders worth 100, 200, 300, 400, and one with NULL. The average of the known values is $(100+200+300+400)/4 = 250$. If we divided by 5, we'd get 200 — implying the unknown order was worth zero, which is wrong.

INTERVIEW CARD

Frequency: Very High

Question: What is the difference between COUNT(*) and COUNT(column)?

Expected Answer: COUNT(*) counts every row in the table including rows with NULL values in any column. COUNT(column) counts only rows where that specific column is NOT NULL — NULLs are skipped. COUNT(DISTINCT column) additionally deduplicates, counting only unique non-NULL values.

Follow-up: If a table has 100 rows and a column has 10 NULLs, what does COUNT() return vs COUNT(col)? (Answer: COUNT() = 100, COUNT(col) = 90.)

PRACTICE QUIZ

1. What does COUNT(*) return for a table with 500 rows where 20 rows have NULL in the 'status' column?
2. Why is AVG(col) not always equal to SUM(col) / COUNT(*)?
3. Predict the output: `SELECT MIN(price), MAX(price) FROM products WHERE category = 'Electronics';` — what does this return if no Electronics products exist?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. 500 — COUNT(*) counts all rows regardless of NULLs.
2. Because AVG excludes NULL rows from both numerator and denominator, but COUNT(*) includes all rows including NULLs.
3. NULL, NULL — aggregate functions return NULL when applied to an empty set.

 CHECKPOINT QUESTION

Write a single query that returns the total number of orders, the total revenue, and the average order value — but only for orders with status 'shipped' or 'delivered' and total_amount IS NOT NULL.

GROUP BY — SPLIT-APPLY-COMBINE AND GROUPING BY ONE OR MULTIPLE COLUMNS

ChatGPT • AI Assistant

A three-stage horizontal diagram illustrating the split-apply-combine model. Stage 1 (left panel, label 'SPLIT — GROUP BY status'): a full orders table with 12 rows is shown. Colored brackets on the right group the rows by status: green bracket for 'delivered' rows, blue for 'shipped', orange for 'pending', red for 'cancelled'. Stage 2 (center panel, label 'APPLY — SUM(total_amount) per group'): each color group now shows a compact sub-table with its rows and a SUM calculation arrow pointing to a single value below each group. Stage 3 (right panel, label 'COMBINE — one output row per group'): a final 4-row result table with columns status and total_revenue, one row per group. Arrows connect each stage. Title above reads: 'GROUP BY = Split → Apply → Combine — identical to Pandas groupby().'

Split-Apply-Combine Mental Model for GROUP BY

sql – GROUP BY Examples – Single Column, Multiple Columns, Expression

```
-- Example 1: Single column GROUP BY
SELECT status,
       COUNT(*)           AS order_count,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status;

-- Example 2: WHERE before GROUP BY
SELECT status,
       COUNT(*)           AS order_count,
       AVG(total_amount) AS avg_value
FROM orders
WHERE status ≠ 'cancelled' -- filter rows FIRST
GROUP BY status;

-- Example 3: Multiple columns GROUP BY
SELECT c.city,
       o.status,
       COUNT(*)           AS order_count,
       SUM(o.total_amount) AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
```

```
GROUP BY c.city, o.status
ORDER BY c.city, revenue DESC;

-- Example 4: GROUP BY with date expression (monthly rollup)
SELECT DATE_TRUNC('month', order_date) AS order_month,
       COUNT(*)                        AS order_count,
       SUM(total_amount)               AS monthly_revenue
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month ASC;

-- Golden Rule violation (FAILS)
SELECT city, name, COUNT(*)
FROM customers
GROUP BY city;           -- ERROR: name not in GROUP BY or aggregate

-- Golden Rule fix
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city;
```

Student

So aggregates collapse all rows into one number. But what if I want the total revenue per city, not just one total for everything?

ChatGPT • AI Assistant

CONCEPT CARD

GROUP BY splits the table into groups first, then applies the aggregate function to each group separately. The result has one row per group.

🔗 ANALOGY

This is identical to what you already know in Pandas:

python – Code Example

```
# Pandas
orders.groupby('status')['total_amount'].sum()

# SQL equivalent
```

```
SELECT status, SUM(total_amount) AS revenue
FROM orders
GROUP BY status;
```

The mental model is **split** → **apply** → **combine** — split the table into groups, apply the aggregate to each group, combine results into one output table.

EXAMPLE CARD

Example 1 — Basic GROUP BY: Revenue per order status

sql — Code Example

```
SELECT status,
       COUNT(*)           AS order_count,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status;
```

Output:

code — Code Example

status	order_count	total_revenue
delivered	1823	2941050.00
shipped	1204	1832600.00
pending	986	1498300.00
cancelled	790	1012400.00

Example 2 — GROUP BY with WHERE: Only non-cancelled orders, grouped by status

sql — Code Example

```
SELECT status,
       COUNT(*)           AS order_count,
       AVG(total_amount) AS avg_value
FROM orders
WHERE status ≠ 'cancelled'
GROUP BY status;
```

WHERE runs BEFORE GROUP BY — it removes cancelled rows first, then grouping happens on the remaining rows.

Example 3 — Multiple columns GROUP BY: Revenue per city per status

sql – Code Example

```
SELECT c.city,
       o.status,
       COUNT(*)           AS order_count,
       SUM(o.total_amount) AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city, o.status
ORDER BY c.city, revenue DESC;
```

Output (sample):

code – Code Example

city	status	order_count	revenue
Delhi	delivered	421	698500.00
Delhi	shipped	302	481200.00
Mumbai	delivered	389	601300.00
...			

Each unique combination of city + status becomes its own group.

Example 4 — GROUP BY with expression: Monthly revenue trend

sql – Code Example

```
SELECT DATE_TRUNC('month', order_date) AS order_month,
       COUNT(*)                       AS order_count,
       SUM(total_amount)               AS monthly_revenue
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month ASC;
```

Output (sample):

code – Code Example

order_month	order_count	monthly_revenue
2024-01-01	312	487500.00

2024-02-01		298		461200.00
2024-03-01		341		529800.00

`DATE_TRUNC('month', order_date)` truncates any date to the first day of its month — this groups all January orders together, all February orders together, and so on.

Student

What is the golden rule I keep hearing about — that every column in SELECT must either be in GROUP BY or inside an aggregate?

ChatGPT • AI Assistant

⚠ WARNING

This is the most important rule in GROUP BY — and the most common mistake beginners make.

The Golden Rule: Every column in SELECT must either:

1. Appear in the GROUP BY clause, OR
2. Be wrapped inside an aggregate function (COUNT, SUM, AVG, MIN, MAX).

⚠ COMMON PITFALL / MISTAKE

✗ Incorrect — name is neither in GROUP BY nor aggregated:

sql – Code Example

```
SELECT city, name, COUNT(*) AS order_count
FROM customers
GROUP BY city;
-- ERROR: column 'name' must appear in GROUP BY or aggregate
```

✓ Correct — include name in GROUP BY:

sql – Code Example

```
SELECT city, name, COUNT(*) AS order_count
FROM customers
GROUP BY city, name;
```

✓ Or aggregate it:

sql — Code Example

```
SELECT city,  
       COUNT(*)      AS customer_count,  
       MIN(name)     AS first_customer_alphabetically  
FROM customers  
GROUP BY city;
```

Why does this rule exist? When you group by city, each group may have many rows with different names. The database cannot know which name to show for the group — it is ambiguous. The rule forces you to either include name in the grouping (so each group has one name) or aggregate it (so you pick one value explicitly).

[Note] MySQL has a non-standard mode that allows this and picks an arbitrary value — avoid relying on it. PostgreSQL and standard SQL enforce the rule strictly.

 INTERVIEW CARD

Frequency: Very High

Question: Explain the GROUP BY golden rule. Why does it exist?

Expected Answer: Every column in SELECT must either appear in GROUP BY or be wrapped in an aggregate function. This rule exists because GROUP BY collapses multiple rows into one per group — if a column is not in GROUP BY, the group may contain many different values for that column, making it ambiguous which value to show. The database enforces the rule to prevent you from accidentally returning meaningless data.

Follow-up: Which database engines enforce this strictly? (Answer: PostgreSQL, SQL Server, Oracle enforce strictly. MySQL in non-strict mode allows it and picks an arbitrary value — dangerous.)

 PRACTICE QUIZ

1. In split-apply-combine: what does GROUP BY do in the 'split' step?
2. Does WHERE run before or after GROUP BY?
3. What error do you get if you add a non-aggregated column to SELECT that is not in GROUP BY?
4. Predict: `SELECT category, MIN(price), MAX(price) FROM products GROUP BY category;` — how many rows does this return if there are 5 distinct categories?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

 ANSWER KEY

1. GROUP BY splits the table into one subset per unique combination of the GROUP BY columns.
2. WHERE runs BEFORE GROUP BY — it filters rows first, then grouping happens on the remaining rows.
3. An error like: 'column must appear in GROUP BY clause or be used in an aggregate function.'
4. 5 rows — one per category.

 CHECKPOINT QUESTION

Write a query that shows the number of customers and the earliest signup date per city, only for cities where customers signed up before 2023. Sort by customer count descending.

HAVING — FILTERING GROUPS AFTER AGGREGATION

ChatGPT • AI Assistant

A vertical flowchart with 5 nodes connected by arrows. Node 1 (orange): 'FROM orders — full table, 4821 rows'. Arrow down. Node 2 (red): 'WHERE status != cancelled — removes 790 rows, 4031 rows remain'. A red callout to the right reads: 'WHERE filters ROWS before grouping.' Arrow down. Node 3 (teal): 'GROUP BY city — splits 4031 rows into N city groups'. Arrow down. Node 4 (purple): 'HAVING SUM(total_amount) > 10000 — removes cities below threshold'. A purple callout to the right reads: 'HAVING filters GROUPS after aggregation. Aggregate functions are available here.' Arrow down. Node 5 (blue): 'SELECT + ORDER BY + LIMIT — final output'. A large red X icon connects 'WHERE COUNT(*) > 5' sideways with label 'SYNTAX ERROR — aggregates not allowed in WHERE.' Clean white background, monospace font for SQL expressions in each node.

WHERE vs HAVING — When Each Filter Runs

sql – HAVING Examples – Filter Groups After Aggregation

```
-- Example 1: Cities with more than 100 orders
SELECT c.city,
       COUNT(*) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
HAVING COUNT(*) > 100
ORDER BY order_count DESC;
```

```
-- Example 2: Categories with total stock value > 50000
SELECT category,
       SUM(price * stock_qty) AS total_stock_value
FROM products
GROUP BY category
HAVING SUM(price * stock_qty) > 50000
ORDER BY total_stock_value DESC;

-- Example 3: WHERE + GROUP BY + HAVING all together
SELECT c.city,
       COUNT(*)           AS order_count,
       SUM(o.total_amount) AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status ≠ 'cancelled'      -- filter rows BEFORE grouping
GROUP BY c.city
HAVING SUM(o.total_amount) > 10000 -- filter groups AFTER aggregating
ORDER BY revenue DESC;

-- WRONG: aggregate in WHERE
-- SELECT city, COUNT(*) FROM orders WHERE COUNT(*) > 100 GROUP BY
city; -- ERROR

-- WRONG: alias in HAVING (PostgreSQL)
-- HAVING order_count > 100; -- ERROR

-- CORRECT: repeat aggregate expression
SELECT city, COUNT(*) AS order_count
FROM orders
GROUP BY city
HAVING COUNT(*) > 100;
```

Student

I can filter rows with WHERE before grouping. But what if I want to filter groups after aggregating — like show only cities with more than 100 orders?

ChatGPT • AI Assistant

CONCEPT CARD

HAVING is the filter that runs **after** GROUP BY and aggregation. WHERE filters rows; HAVING filters groups.

sql – Code Example

```
SELECT city, COUNT(*) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY city
HAVING COUNT(*) > 100;
```

🔗 ANALOGY

WHERE is the bouncer at the door who checks everyone before the party (rows).
HAVING is the manager inside who reviews each table group and removes the ones that don't meet criteria (groups).

EXAMPLE CARD

Example 1 — Basic HAVING: Cities with more than 100 orders

sql – Code Example

```
SELECT c.city,
       COUNT(*) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
HAVING COUNT(*) > 100
ORDER BY order_count DESC;
```

Output:

code – Code Example

city	order_count
Mumbai	842
Delhi	731
Bangalore	489
Hyderabad	312

Cities with 100 or fewer orders are excluded from the result.

Example 2 — HAVING with SUM: Categories with total stock value above ₹50,000

sql – Code Example

```
SELECT category,
       SUM(price * stock_qty) AS total_stock_value
FROM products
GROUP BY category
HAVING SUM(price * stock_qty) > 50000
ORDER BY total_stock_value DESC;
```

Example 3 — WHERE and HAVING together (the critical distinction)

sql – Code Example

```
SELECT c.city,
       COUNT(*)           AS order_count,
       SUM(o.total_amount) AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status ≠ 'cancelled'           -- Step 2: filter rows BEFORE
grouping
GROUP BY c.city                       -- Step 3: group remaining
rows
HAVING SUM(o.total_amount) > 10000     -- Step 4: filter groups AFTER
aggregating
ORDER BY revenue DESC;
```

Explanation:

- **WHERE o.status ≠ 'cancelled'** removes cancelled order rows before any grouping happens.
- **HAVING SUM(o.total_amount) > 10000** then removes cities whose non-cancelled revenue is under ₹10,000 after aggregation.

❗ COMMON PITFALL / MISTAKE**✗ Incorrect — using WHERE to filter on an aggregate:**

sql – Code Example

```
SELECT city, COUNT(*) AS order_count
FROM orders
WHERE COUNT(*) > 100    -- ERROR: aggregate functions not allowed in
WHERE
GROUP BY city;
```

 Correct — use HAVING for aggregate conditions:

sql – Code Example

```
SELECT city, COUNT(*) AS order_count
FROM orders
GROUP BY city
HAVING COUNT(*) > 100;
```

 WARNING

Can you use a SELECT alias in HAVING? No — HAVING runs at step 4, before SELECT at step 5. Repeat the aggregate expression in HAVING.

 COMMON PITFALL / MISTAKE **Alias in HAVING — does NOT work in standard SQL:**

sql – Code Example

```
SELECT city, COUNT(*) AS order_count
FROM orders
GROUP BY city
HAVING order_count > 100;    -- ERROR in PostgreSQL, Oracle, SQL Server
```

 Correct — repeat the aggregate:

sql – Code Example

```
SELECT city, COUNT(*) AS order_count
FROM orders
GROUP BY city
HAVING COUNT(*) > 100;
```

Note: MySQL and some databases allow aliases in HAVING as an extension — but this is non-standard and not portable.

 INTERVIEW CARD**Frequency:** Very High**Question:** What is the difference between WHERE and HAVING?

Expected Answer: WHERE filters individual rows BEFORE grouping — it runs at step 2 of execution. HAVING filters groups AFTER aggregation — it runs at step 4. You cannot use aggregate functions (COUNT, SUM, AVG, etc.) in a WHERE clause — that

is a syntax error. You must use HAVING for any condition that involves an aggregate result. A query can use both: WHERE to pre-filter rows before grouping, and HAVING to post-filter groups after aggregation.

Follow-up: Can you always replace HAVING with a WHERE in a subquery? (Answer: Yes — wrap the GROUP BY query in a subquery and apply WHERE on the outer query. But HAVING is cleaner and more readable.)

🔍 PRACTICE QUIZ

1. What is the execution order step for HAVING? What runs before and after it?
2. Why can you NOT write `WHERE COUNT(*) > 5` ?
3. A query groups orders by *customerid* and computes *SUM(totalamount)*. Write the HAVING clause to keep only customers who spent more than ₹5000 total.
4. Predict: does `HAVING order_count > 10` work if order_count is a SELECT alias? (Assume PostgreSQL.)

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. HAVING is step 4. GROUP BY (step 3) runs before it. SELECT (step 5) runs after it.
2. Because WHERE runs at step 2 — before GROUP BY and aggregation at step 3. The aggregate result doesn't exist yet.
3. `HAVING SUM(total_amount) > 5000`
4. No — in PostgreSQL, HAVING runs before SELECT so aliases are not visible. Must write `HAVING COUNT(*) > 10` .

🔍 CHECKPOINT QUESTION

Write a query that shows each product category, the number of products in it, and the average price — but only for categories that have at least 3 products AND an average price above ₹500. Sort by average price descending.

COUNT DISTINCT, DATE_TRUNC, AND THE FULL EXECUTION ORDER

A vertical flowchart with 7 numbered rectangular nodes, each a different color, connected by downward arrows. Node 1 (orange): 'Step 1 — FROM: identify the table(s). All rows available.' Node 2 (red): 'Step 2 — WHERE: filter individual rows. Aggregates NOT allowed here.' Node 3 (teal): 'Step 3 — GROUP BY: split remaining rows into groups per unique column combination.' Node 4 (purple): 'Step 4 — HAVING: filter groups. Aggregates ARE allowed here.' Node 5 (blue): 'Step 5 — SELECT: compute output columns and aliases.' Node 6 (dark blue): 'Step 6 — ORDER BY: sort the result. Aliases ARE visible here.' Node 7 (green): 'Step 7 — LIMIT: cap the number of output rows.' Two annotation boxes on the right: red box reads 'WHERE = row filter (pre-group). HAVING = group filter (post-aggregate).' Green box reads 'Mnemonic: F-W-G-H-S-O-L — Fried Wings Get Hot Sometimes On Lunch.' Clean white background, monospace SQL examples inside each node in smaller text.

Full 7-Step SQL Execution Order — Complete Reference

sql — COUNT DISTINCT, DATE_TRUNC, and Full 7-Clause Query

```
-- COUNT(DISTINCT) inside GROUP BY
SELECT c.city,
       COUNT(DISTINCT o.customer_id) AS unique_customers,
       COUNT(*)                      AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY unique_customers DESC;

-- Monthly revenue trend using DATE_TRUNC
SELECT DATE_TRUNC('month', order_date) AS order_month,
       COUNT(*)                      AS total_orders,
       COUNT(DISTINCT customer_id)   AS unique_customers,
       SUM(total_amount)              AS monthly_revenue,
       AVG(total_amount)              AS avg_order_value
FROM orders
WHERE status IN ('shipped', 'delivered')
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month ASC;

-- Full query using all 7 clauses
SELECT c.city,
       DATE_TRUNC('month', o.order_date) AS order_month,
       COUNT(*)                      AS order_count,
       COUNT(DISTINCT o.customer_id) AS unique_buyers,
       SUM(o.total_amount)           AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status IN ('shipped', 'delivered')
GROUP BY c.city, DATE_TRUNC('month', o.order_date)
HAVING SUM(o.total_amount) > 5000
```

```
ORDER BY revenue DESC
LIMIT 10;
```

Student

Can I count unique values inside a GROUP BY? Like, how many distinct customers placed orders in each city?

ChatGPT • AI Assistant

CONCEPT CARD

`COUNT(DISTINCT column)` counts unique non-NULL values within each group. You can combine it with any GROUP BY.

sql – Code Example

```
SELECT c.city,
       COUNT(DISTINCT o.customer_id) AS unique_customers,
       COUNT(*)                      AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city;
```

Output:

code – Code Example

city	unique_customers	total_orders
Mumbai	312	842
Delhi	278	731
Bangalore	189	489

`unique_customers` counts how many distinct customer IDs appear in each city — a customer who placed 5 orders is counted once, not 5 times.

EXAMPLE CARD

Example 1 — COUNT(DISTINCT) for unique product categories ordered per month

sql – Code Example

```
SELECT DATE_TRUNC('month', o.order_date) AS order_month,  
       COUNT(DISTINCT p.category)        AS unique_categories  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN products p    ON oi.product_id = p.product_id  
GROUP BY DATE_TRUNC('month', o.order_date)  
ORDER BY order_month;
```

Example 2 — Monthly revenue trend (a real analyst query)

sql – Code Example

```
SELECT DATE_TRUNC('month', order_date) AS order_month,  
       COUNT(*)                        AS total_orders,  
       COUNT(DISTINCT customer_id)    AS unique_customers,  
       SUM(total_amount)               AS monthly_revenue,  
       AVG(total_amount)               AS avg_order_value  
FROM orders  
WHERE status IN ('shipped', 'delivered')  
GROUP BY DATE_TRUNC('month', order_date)  
ORDER BY order_month ASC;
```

This is a genuine business intelligence query — product managers and data analysts write exactly this to produce monthly KPI reports.

Student

Can you give me the full execution order now that we have GROUP BY and HAVING?

ChatGPT • AI Assistant

CONCEPT CARD

Here is the complete execution order for a query with all clauses:

Step 1: FROM

Step 2: WHERE

Step 3: GROUP\ BY

Step 4: HAVING

Step 5: SELECT

Step 6: ORDER\ BY

Step 7: LIMIT

MEMORY TRICK CARD

Expanded mnemonic: **F-W-G-H-S-O-L**

Fried **W**ings **G**et **H**ot **S**ometimes **O**n **L**unch

= FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

EXAM TIP CARD

Use this table to decide WHERE vs HAVING for any condition:

- Condition uses a raw column value? → **WHERE**
- Condition uses an aggregate result? → **HAVING**
- When in doubt: can this condition run before rows are grouped? → **WHERE**. Can it only run after grouping? → **HAVING**.

EXAMPLE CARD

Example 3 — Full query using all seven clauses

sql – Code Example

```
SELECT c.city,
       DATE_TRUNC('month', o.order_date) AS order_month,
       COUNT(*)                          AS order_count,
       COUNT(DISTINCT o.customer_id)     AS unique_buyers,
       SUM(o.total_amount)                AS revenue
FROM   orders o
JOIN   customers c ON o.customer_id = c.customer_id
WHERE  o.status IN ('shipped', 'delivered') -- Step 2
GROUP BY c.city,
         DATE_TRUNC('month', o.order_date) -- Step 3
HAVING SUM(o.total_amount) > 5000          -- Step 4
ORDER BY revenue DESC                     -- Step 6
LIMIT 10;                                -- Step 7
```

INTERVIEW CARD

Frequency: High

Question: What is the full SQL logical execution order including GROUP BY and HAVING?

Expected Answer: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. WHERE filters rows before grouping. GROUP BY splits rows into groups. HAVING filters groups after aggregation. SELECT computes output columns. ORDER BY sorts. LIMIT caps the result. This order explains why aggregate functions cannot appear in WHERE (aggregation hasn't happened yet) and why SELECT aliases cannot be used in WHERE or HAVING (SELECT runs after both).

Follow-up: Can you use a SELECT alias in ORDER BY? (Answer: Yes — ORDER BY runs after SELECT.)

🔗 PRACTICE QUIZ

1. Write the full 7-step SQL execution order from memory.
2. What does `COUNT(DISTINCT customer_id)` return in a GROUP BY query?
3. What function truncates a date to the first day of its month in PostgreSQL?
4. Predict the output row count: a query groups orders by month and by status. There are 12 months and 4 statuses. How many output rows maximum?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
2. The number of unique customer IDs within each group — a customer who placed multiple orders is counted once per group.
3. `DATE_TRUNC('month', date_column)`
4. Maximum 48 rows (12 months × 4 statuses) — fewer if some month-status combinations have no orders.

🔗 CHECKPOINT QUESTION

Before moving to SQL 1.3 (Joins): write a single query that reports the monthly revenue trend for 2024 only, for shipped or delivered orders, showing months where total revenue exceeded ₹50,000, including the count of unique customers who ordered that month. Sort by month ascending.

10 PRACTICE PROBLEMS — AGGREGATIONS AND GROUP BY

ChatGPT • AI Assistant

A decision tree flowchart. The root node (white box) reads: 'I want to filter data in a GROUP BY query.' First decision diamond: 'Does the condition involve an aggregate function like COUNT, SUM, AVG, MIN, MAX?' Left branch (NO, green): leads to a green box labeled 'Use WHERE — filter rows before grouping. Example: WHERE status != cancelled.' Right branch (YES, orange): leads to an orange box labeled 'Use HAVING — filter groups after aggregation. Example: HAVING COUNT(*) > 100.' Below both boxes, a third box (blue) reads: 'Can I use BOTH in the same query? YES — WHERE pre-filters rows, then HAVING post-filters groups.' A fourth red warning box reads: 'NEVER write WHERE COUNT(*) > N — this is always a syntax error.' Clean flowchart style with rounded decision diamonds, rectangular action boxes, and color-coded branches.

WHERE vs HAVING Decision Flowchart

sql — 10 Practice Problems — Complete Solutions

```
-- Problem 1: Total orders
SELECT COUNT(*) AS total_orders FROM orders;

-- Problem 2: Revenue and avg order value
SELECT SUM(total_amount) AS total_revenue,
       AVG(total_amount) AS avg_order_value
FROM orders;

-- Problem 3: Products per category
SELECT category, COUNT(*) AS product_count
FROM products GROUP BY category ORDER BY product_count DESC;

-- Problem 4: Price range per category
SELECT category, MIN(price) AS cheapest, MAX(price) AS most_expensive
FROM products GROUP BY category;

-- Problem 5: Orders and revenue per status (no cancelled)
SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS
total_revenue
FROM orders WHERE status ≠ 'cancelled'
GROUP BY status ORDER BY total_revenue DESC;

-- Problem 6: Cities with more than 50 customers
SELECT city, COUNT(*) AS customer_count
FROM customers GROUP BY city
HAVING COUNT(*) > 50 ORDER BY customer_count DESC;

-- Problem 7: Avg order value per city > 1000 (shipped/delivered only)
SELECT c.city, AVG(o.total_amount) AS avg_order_value
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status IN ('shipped','delivered')
```

```
GROUP BY c.city HAVING AVG(o.total_amount) > 1000
ORDER BY avg_order_value DESC;

-- Problem 8: Stock qty and value per category
SELECT category,
       SUM(stock_qty) AS total_units,
       SUM(price * stock_qty) AS total_stock_value
FROM products GROUP BY category ORDER BY total_stock_value DESC;

-- Problem 9: Monthly revenue 2024, delivered only, revenue > 100000
SELECT DATE_TRUNC('month', order_date) AS order_month,
       COUNT(*) AS order_count, SUM(total_amount) AS monthly_revenue
FROM orders
WHERE status = 'delivered'
      AND order_date ≥ '2024-01-01' AND order_date < '2025-01-01'
GROUP BY DATE_TRUNC('month', order_date)
HAVING SUM(total_amount) > 100000
ORDER BY order_month ASC;

-- Problem 10: City x month trend 2024, revenue > 5000 and ≥ 10 unique
customers
SELECT c.city,
       DATE_TRUNC('month', o.order_date) AS order_month,
       COUNT(*) AS order_count,
       COUNT(DISTINCT o.customer_id) AS unique_customers,
       SUM(o.total_amount) AS revenue
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status IN ('shipped', 'delivered')
      AND o.order_date ≥ '2024-01-01' AND o.order_date < '2025-01-01'
GROUP BY c.city, DATE_TRUNC('month', o.order_date)
HAVING SUM(o.total_amount) > 5000 AND COUNT(DISTINCT o.customer_id) ≥
10
ORDER BY revenue DESC LIMIT 10;
```

Student

Can I try some practice problems to put it all together?

ChatGPT • AI Assistant

CONCEPT CARD

Here are 10 problems of increasing difficulty — all using the same e-commerce schema. Try each one yourself before looking at the solution.

Problem 1 — Easy: Total number of orders in the orders table.

sql – Code Example

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Problem 2 — Easy: Total revenue and average order value across all orders.

sql – Code Example

```
SELECT SUM(total_amount) AS total_revenue,
       AVG(total_amount) AS avg_order_value
FROM orders;
```

Problem 3 — Easy: Number of products per category.

sql – Code Example

```
SELECT category,
       COUNT(*) AS product_count
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

Problem 4 — Easy: Minimum and maximum product price per category.

sql – Code Example

```
SELECT category,
       MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products
GROUP BY category;
```

Problem 5 — Medium: Number of orders and total revenue per order status, excluding cancelled orders.

sql – Code Example

```
SELECT status,  
       COUNT(*)           AS order_count,  
       SUM(total_amount)  AS total_revenue  
FROM orders  
WHERE status ≠ 'cancelled'  
GROUP BY status  
ORDER BY total_revenue DESC;
```

Problem 6 — Medium: Cities with more than 50 customers. Show city and customer count.

sql – Code Example

```
SELECT city,  
       COUNT(*) AS customer_count  
FROM customers  
GROUP BY city  
HAVING COUNT(*) > 50  
ORDER BY customer_count DESC;
```

Problem 7 — Medium: Average order value per city, only for shipped or delivered orders, only for cities with average order value above ₹1000.

sql – Code Example

```
SELECT c.city,  
       AVG(o.total_amount) AS avg_order_value  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
WHERE o.status IN ('shipped', 'delivered')  
GROUP BY c.city  
HAVING AVG(o.total_amount) > 1000  
ORDER BY avg_order_value DESC;
```

Problem 8 — Medium: Total stock quantity and total stock value (price × stock_qty) per category.

sql – Code Example

```
SELECT category,  
       SUM(stock_qty)           AS total_units,  
       SUM(price * stock_qty)  AS total_stock_value  
FROM products
```

```
GROUP BY category
ORDER BY total_stock_value DESC;
```

Problem 9 — Hard: Monthly order count and revenue for the year 2024, only for delivered orders, only for months where revenue exceeded ₹100,000. Sort by month ascending.

sql — Code Example

```
SELECT DATE_TRUNC('month', order_date) AS order_month,
       COUNT(*)                        AS order_count,
       SUM(total_amount)                AS monthly_revenue
FROM orders
WHERE status = 'delivered'
  AND order_date ≥ '2024-01-01'
  AND order_date < '2025-01-01'
GROUP BY DATE_TRUNC('month', order_date)
HAVING SUM(total_amount) > 100000
ORDER BY order_month ASC;
```

Problem 10 — Hard: Monthly revenue trend — city, month, total orders, unique customers, and total revenue — for shipped or delivered orders in 2024, only for city-month combinations with revenue above ₹5000 and at least 10 unique customers. Top 10 by revenue descending.

sql — Code Example

```
SELECT c.city,
       DATE_TRUNC('month', o.order_date) AS order_month,
       COUNT(*)                        AS order_count,
       COUNT(DISTINCT o.customer_id)   AS unique_customers,
       SUM(o.total_amount)              AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status IN ('shipped', 'delivered')
  AND o.order_date ≥ '2024-01-01'
  AND o.order_date < '2025-01-01'
GROUP BY c.city,
         DATE_TRUNC('month', o.order_date)
HAVING SUM(o.total_amount) > 5000
  AND COUNT(DISTINCT o.customer_id) ≥ 10
ORDER BY revenue DESC
LIMIT 10;
```

SUMMARY CARD

SQL 1.2 Key Takeaways:

- ***COUNT() = all rows. COUNT(col) = non-NULL rows. COUNT(DISTINCT col)** = unique non-NULL values.**
- **SUM, AVG, MIN, MAX** all ignore NULLs — be careful with AVG when NULLs are present.
- **GROUP BY** splits rows into groups. The result has one row per unique group.
- **The Golden Rule:** every SELECT column must be in GROUP BY or wrapped in an aggregate.
- **WHERE** filters rows BEFORE grouping. Cannot use aggregates in WHERE.
- **HAVING** filters groups AFTER aggregation. Can use aggregates in HAVING.
- **DATE_TRUNC('month', col)** groups dates by month (PostgreSQL).
- **COUNT(DISTINCT col)** inside GROUP BY counts unique values per group.
- Full execution order: **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.**
- Mnemonic: **F-W-G-H-S-O-L** — Fried Wings Get Hot Sometimes On Lunch.

✧ AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned GROUP BY, HAVING, COUNT DISTINCT, and DATE_TRUNC. Give me 10 progressively harder GROUP BY problems using an e-commerce schema with tables: customers(customerid, name, city, signupdate), orders(orderid, customerid, orderdate, totalamount, status), products(productid, name, category, price, stock_qty). After each problem: wait for my answer, then show the correct SQL, explain every clause, and show one common mistake students make on that problem. Do not give me the next problem until I attempt the current one.'

🔗 CHECKPOINT QUESTION

Before moving to SQL 1.3 (Joins): without running it, identify every mistake in this query and write the fully corrected version:

sql — Code Example

```
SELECT city, name, COUNT(*) AS orders, AVG(total_amount) AS avg_val
FROM orders
WHERE COUNT(*) > 5
GROUP BY city
```

```
HAVING avg_val > 1000  
ORDER BY avg_val DESC;
```

✧ CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.2 — aggregations and group by, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **SQL Full Logical Execution Order with GROUP BY and HAVING:** A vertical flowchart showing 7 numbered steps in the SQL logical execution order...